

Introduction to the Project

The Hotel Management System (HMS) is a basic program that assists in overseeing hotel tasks such as guest registration, room reservations, and invoicing. Python is utilized for programming, while SQLite is employed for the database.

Main Characteristics:

Register customers: Record customer information such as name, address, and contact details.

Booking rooms and determining room fees will be done by Room Booking, taking into account the room category and length of stay.

Bill Management: Keep track of and compute costs for various services such as dining expenses, gaming transactions, and fashion retail buys.

Total Bill Calculation: Determine the total amount due by adding up all charges for a customer.

Search for customers and access their details as well as their booking history.

Structure of a database:

Customer Details (C_DETAILS): Records details of customers.

Room Rent (ROOM_RENT): Maintains records of room reservations and costs.

Bills for services (RESTAURANT_BILL, GAMING_BILL, FASHION_BILL): Fees for various types of services provided by the store.

TOTAL_BILL is where the complete payment for every client is recorded.

main.py (Code)

```
import sqlite3
from datetime import datetime
from Backends.db_operations import initialize_database, get_connection

# GLOBAL SYSTEM TRACKING VARIABLES
roomRent = 0
restaurantBill = 0
gamingBill = 0
fashionBill = 0
cid = ""
new_customer_added = False

def userEntry():
    global cid, new_customer_added
    conn = get_connection()
    cursor = conn.cursor()

    while True:
        print("\n1. Add New Customer")
        print("9. Exit")
        choice = input("\nEnter your choice: ").strip()

        if choice == '1':
            cid = input("Enter Customer Identification Number: ").strip()
            if not cid.strip():
                print("Customer ID is required. Please enter a valid ID.")
                continue

            cursor.execute("SELECT CID FROM C_DETAILS WHERE CID = ?", (cid,))
            if cursor.fetchone():
                print("Customer ID already exists. Please enter a unique ID.")
            else:
                break
        elif choice == '9':
            print("Exiting entry module.")
            conn.close()
            return
        else:
            print("Invalid choice!")

    name = input("Enter Customer Name: ")
    address = input("Enter Customer Address: ")
```

```

age = input("Enter Customer Age: ")
nationality = input("Enter Customer Country: ")
phoneno = input("Enter Customer Contact Number: ") # Saved safely as string text
email = input("Enter Customer Email: ")

cursor.execute(
    "INSERT INTO C_DETAILS (CID, C_NAME, C_ADDRESS, C_AGE, C_COUNTRY,
P_NO, C_EMAIL) VALUES (?, ?, ?, ?, ?, ?, ?)",
    (cid, name, address, age, nationality, phoneno, email)
)
conn.commit()
print("\nNew Customer entered in the system successfully!")
new_customer_added = True
conn.close()

def get_customer_id(prompt_message):
    """Helper function to look up customer records safely."""
    global cid, new_customer_added
    if new_customer_added:
        print("\nUsing active customer session data, skipping lookup prompt.")
        return True

    print(prompt_message)
    customer_detail = input("Enter Customer ID or Name: ").strip()

    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT CID FROM C_DETAILS WHERE CID = ? OR C_NAME = ?",
(customer_detail, customer_detail))
    customer = cursor.fetchone()
    conn.close()

    if customer:
        cid = customer[0]
        return True
    else:
        print("Customer record not found inside the database!")
        return False

def bookingRecord():
    global cid, roomRent, new_customer_added
    if not get_customer_id("Enter customer details to make a booking."):
        return

    print("\n##### We have the following rooms for you #####")
    print("1. Ultra Royal > 10000 Rs. per day")

```

```

print("2. Royal > 5000 Rs. per day")
print("3. Elite > 3500 Rs. per day")
print("4. Budget > 2500 Rs. per day")
print("5. Exit")

try:
    room_choice = int(input("Enter your choice: "))
except ValueError:
    print("Invalid input! Please enter a number.")
    return

if room_choice == 5:
    return

start_date = input("Enter Start Date (YYYY/MM/DD): ")
end_date = input("Enter End Date (YYYY/MM/DD): ")

try:
    start_date_obj = datetime.strptime(start_date, '%Y/%m/%d')
    end_date_obj = datetime.strptime(end_date, '%Y/%m/%d')
    days_of_stay = (end_date_obj - start_date_obj).days

    if days_of_stay <= 0:
        print("Invalid date range. End date must be after start date.")
        return
except ValueError:
    print("Invalid date format. Use YYYY/MM/DD.")
    return

rates = {1: 10000, 2: 5000, 3: 3500, 4: 2500}
names = {1: "Ultra Royal", 2: "Royal", 3: "Elite", 4: "Budget"}

if room_choice in rates:
    roomRent = days_of_stay * rates[room_choice]
    room_name = names[room_choice]
else:
    print("Invalid Choice.")
    return

conn = get_connection()
cursor = conn.cursor()
cursor.execute(
    "INSERT INTO ROOM_BOOKINGS (CID, ROOM_CHOICE, START_DATE,
    END_DATE, TOTAL_PRICE) VALUES (?, ?, ?, ?, ?)",
    (cid, room_name, start_date, end_date, roomRent)
)

```

```

conn.commit()
conn.close()
print(f"Room successfully booked for {days_of_stay} days. Total: Rs. {roomRent}")
new_customer_added = False # Reset flag for clean future option lookups

def calculateRoomRent():
    global cid, roomRent, new_customer_added
    if not get_customer_id("Enter customer details to calculate room rent."):
        return

    print("\n##### We have the following rooms for you #####")
    print("1. Ultra Royal > 10000 Rs.")
    print("2. Royal > 5000 Rs.")
    print("3. Elite > 3500 Rs.")
    print("4. Budget > 2500 Rs.")
    print("5. Exit")

    try:
        room_choice = int(input("Enter your choice: "))
        if room_choice == 5:
            return

        room_no = input("Enter Customer Room No: ").strip()
        no_of_days = int(input("Enter No. of Days: "))
    except ValueError:
        print("Invalid numeric entry! Process terminated safely.")
        return

    rates = {1: 10000, 2: 5000, 3: 3500, 4: 2500}
    names = {1: "Ultra Royal", 2: "Royal", 3: "Elite", 4: "Budget"}

    if room_choice in rates:
        roomRent = no_of_days * rates[room_choice]
        room_name = names[room_choice]
    else:
        print("Invalid Choice.")
        return

    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO ROOM_RENT (CID, ROOM_CHOICE, NO_OF_DAYS, ROOM_NO, ROOM_RENT) VALUES (?, ?, ?, ?, ?)",
        (cid, room_name, no_of_days, room_no, roomRent)
    )
    conn.commit()

```

```

conn.close()
print(f"Room charges calculated successfully. Rent: Rs. {roomRent}")
new_customer_added = False

def restaurant():
    global cid, restaurantBill, new_customer_added
    if not get_customer_id("Enter customer details for restaurant bill."):
        return

    print("\n1. Vegetarian Combo > 300 Rs.")
    print("2. Non-Vegetarian Combo > 500 Rs.")
    print("3. Vegetarian & Non-Vegetarian Combo > 750 Rs.")
    print("4. Exit")

    try:
        choice_dish = int(input("Enter your choice: "))
        if choice_dish == 4:
            return
        quantity = int(input("Enter Quantity: "))
    except ValueError:
        print("Invalid input! Numbers only.")
        return

    rates = {1: 300, 2: 500, 3: 750}
    names = {1: "Vegetarian Combo", 2: "Non-Vegetarian Combo", 3: "Veg & Non-Veg Combo"}

    if choice_dish in rates:
        restaurantBill = quantity * rates[choice_dish]
        dish_name = names[choice_dish]
    else:
        print("Invalid choice!")
        return

    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO RESTAURANT (CID, CUISINE, QUANTITY, BILL) VALUES (?, ?, ?, ?)",
        (cid, dish_name, quantity, restaurantBill)
    )
    conn.commit()
    conn.close()
    print(f"Restaurant invoice updated: Rs. {restaurantBill}")
    new_customer_added = False

def gaming():

```

```

global cid, gamingBill, new_customer_added
if not get_customer_id("Enter customer details for gaming bill."):
    return

print("\n1. Virtual Reality > 1500 Rs.")
print("2. Arcade > 1000 Rs.")
print("3. Racing > 2000 Rs.")
print("4. Escape Room > 2500 Rs.")
print("5. Exit")

try:
    choice_game = int(input("Enter your choice: "))
    if choice_game == 5:
        return
    game_time = int(input("Enter Game Time (in hours): "))
except ValueError:
    print("Invalid input! Numbers only.")
    return

rates = {1: 1500, 2: 1000, 3: 2000, 4: 2500}
names = {1: "Virtual Reality", 2: "Arcade", 3: "Racing", 4: "Escape Room"}

if choice_game in rates:
    gamingBill = game_time * rates[choice_game]
    game_name = names[choice_game]
else:
    print("Invalid choice!")
    return

conn = get_connection()
cursor = conn.cursor()
cursor.execute(
    "INSERT INTO GAMING (CID, GAME_NAME, GAME_TIME, BILL) VALUES (?, ?, ?, ?)",
    (cid, game_name, game_time, gamingBill)
)
conn.commit()
conn.close()
print(f"Gaming invoice updated: Rs. {gamingBill}")
new_customer_added = False

def fashionCounterBill():
    global cid, fashionBill, new_customer_added
    if not get_customer_id("Enter customer details for fashion bill."):
        return

```

```
print("\n1. T-Shirt > 500 Rs.")
print("2. Jeans > 1000 Rs.")
print("3. Jacket > 1500 Rs.")
print("4. Shoes > 2000 Rs.")
print("5. Exit")
```

```
try:
    choice_item = int(input("Enter your choice: "))
    if choice_item == 5:
        return
    quantity = int(input("Enter Quantity: "))
except ValueError:
    print("Invalid input! Numbers only.")
    return
```

```
rates = {1: 500, 2: 1000, 3: 1500, 4: 2000}
names = {1: "T-Shirt", 2: "Jeans", 3: "Jacket", 4: "Shoes"}
```

```
if choice_item in rates:
    fashionBill = quantity * rates[choice_item]
    item_name = names[choice_item]
else:
    print("Invalid choice!")
    return
```

```
conn = get_connection()
cursor = conn.cursor()
cursor.execute(
    "INSERT INTO FASHION (CID, ITEM_NAME, QUANTITY, BILL) VALUES (?, ?, ?,
?)",
    (cid, item_name, quantity, fashionBill)
)
conn.commit()
conn.close()
print(f"Retail shop bill updated: Rs. {fashionBill}")
new_customer_added = False
```

```
def showTotalBill():
    global cid, new_customer_added

    print("Enter customer details to show total bill.")
    customer_detail = input("Enter Customer ID or Name: ").strip()

    conn = get_connection()
    cursor = conn.cursor()
```



```

    cursor.execute("SELECT CID FROM C_DETAILS WHERE CID = ? OR C_NAME = ?",
(customer_detail, customer_detail))
    customer = cursor.fetchone()

    if not customer:
        print("Customer not found!")
        conn.close()
        return

    cid = customer[0]

    # Logical Bug Fix: Uses SUM() to calculate aggregates perfectly for multi-purchases
    cursor.execute("SELECT SUM(ROOM_RENT) FROM ROOM_RENT WHERE CID = ?",
(cid,))
    res = cursor.fetchone()
    room_total = res[0] if res[0] is not None else 0

    cursor.execute("SELECT SUM(BILL) FROM RESTAURANT WHERE CID = ?", (cid,))
    res = cursor.fetchone()
    food_total = res[0] if res[0] is not None else 0

    cursor.execute("SELECT SUM(BILL) FROM GAMING WHERE CID = ?", (cid,))
    res = cursor.fetchone()
    game_total = res[0] if res[0] is not None else 0

    cursor.execute("SELECT SUM(BILL) FROM FASHION WHERE CID = ?", (cid,))
    res = cursor.fetchone()
    shop_total = res[0] if res[0] is not None else 0

    grand_total = room_total + food_total + game_total + shop_total

    print("\n\n***** FINAL BILL *****")
    print("Customer ID:", cid)
    print("=====")
    print(f"Room Rent:      Rs. {room_total}")
    print(f"Restaurant Bill: Rs. {food_total}")
    print(f"Gaming Bill:     Rs. {game_total}")
    print(f"Fashion Bill:    Rs. {shop_total}")
    print("=====")
    print(f"Total Amount Due: Rs. {grand_total}")
    print("*****\n")
    conn.close()
    new_customer_added = False

def searchCustomer():
    print("Search customer entry panel.")

```

```

customer_detail = input("Enter Customer ID or Name: ").strip()

conn = get_connection()
cursor = conn.cursor()
cursor.execute("SELECT * FROM C_DETAILS WHERE CID = ? OR C_NAME = ?",
(customer_detail, customer_detail))
customer = cursor.fetchone()
conn.close()

if customer:
    print("\nCustomer record discovered:")
    print(f"ID:          {customer[0]}")
    print(f"Name:         {customer[1]}")
    print(f"Address:       {customer[2]}")
    print(f"Age:          {customer[3]}")
    print(f"Country:      {customer[4]}")
    print(f"Contact Number: {customer[5]}")
    print(f"Email:        {customer[6]}")
else:
    print("No matching profiles discovered.")

def main():
    initialize_database()
    while True:
        print("""
1. Add New Customer
2. Make a Booking
3. Room Rent
4. Restaurant Bill
5. Gaming Bill
6. Fashion Bill
7. Show Total Bill
8. Search Customer
9. Exit
""")
        raw_choice = input("\nEnter your choice: ").strip()

        try:
            choice = int(raw_choice)
        except ValueError:
            print("\n[!] Invalid input choice! Please select a valid option (1-9).")
            continue

        if choice == 1:
            userEntry()
        elif choice == 2:

```

```
    bookingRecord()
elif choice == 3:
    calculateRoomRent()
elif choice == 4:
    restaurant()
elif choice == 5:
    gaming()
elif choice == 6:
    fashionCounterBill()
elif choice == 7:
    showTotalBill()
elif choice == 8:
    searchCustomer()
elif choice == 9:
    print("\nThank you for using the system!")
    break
else:
    print("\nInvalid selection! Select numbers 1 through 9.")

if __name__ == "__main__":
    main()
```

db_operations.py (Code)

```
import sqlite3
```

```
DB_NAME = "HMS.db"
```

```
def get_connection():
```

```
    """Establishes an independent relational link to the project SQLite file."""
```

```
    return sqlite3.connect(DB_NAME)
```

```
def initialize_database():
```

```
    """Initializes the SQLite master database structure and relational tables."""
```

```
    conn = get_connection()
```

```
    cursor = conn.cursor()
```

```
# Master Guest Profiles Table (P_NO changed to TEXT)
```

```
cursor.execute('''
```

```
    CREATE TABLE IF NOT EXISTS C_DETAILS (
```

```
        CID TEXT PRIMARY KEY,
```

```
        C_NAME TEXT,
```

```
        C_ADDRESS TEXT,
```

```
        C_AGE INT,
```

```
        C_COUNTRY TEXT,
```

```
        P_NO TEXT,
```

```
        C_EMAIL TEXT
```

```
    )
```

```
''')
```

Room Calendar Reservations Table

```
cursor.execute('''  
    CREATE TABLE IF NOT EXISTS ROOM_BOOKINGS (  
        CID TEXT,  
        ROOM_CHOICE TEXT,  
        START_DATE TEXT,  
        END_DATE TEXT,  
        TOTAL_PRICE INT,  
        FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)  
    )  
''')
```

Active Room Rentals Table

```
cursor.execute('''  
    CREATE TABLE IF NOT EXISTS ROOM_RENT (  
        CID TEXT,  
        ROOM_CHOICE TEXT,  
        NO_OF_DAYS INT,  
        ROOM_NO TEXT,  
        ROOM_RENT INT,  
        FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)  
    )  
''')
```

Food & Cuisine Orders Table

```
cursor.execute('''
```

```
CREATE TABLE IF NOT EXISTS RESTAURANT (
```

```
    CID TEXT,
```

```
    CUISINE TEXT,
```

```
    QUANTITY INT,
```

```
    BILL INT,
```

```
    FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)
```

```
)
```

```
''')
```

Entertainment & Arcade Logging Table

```
cursor.execute('''
```

```
CREATE TABLE IF NOT EXISTS GAMING (
```

```
    CID TEXT,
```

```
    GAME_NAME TEXT,
```

```
    GAME_TIME INT,
```

```
    BILL INT,
```

```
    FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)
```

```
)
```

```
''')
```

Apparel Counter Purchases Table

```
cursor.execute('''
```

```
CREATE TABLE IF NOT EXISTS FASHION (
```

```
    CID TEXT,
```

```
    ITEM_NAME TEXT,
```

QUANTITY INT,

BILL INT,

FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)

)

''')

conn.commit()

conn.close()

print("\nCONGRATULATIONS! Your SQLite database connections have been
established!")

Output Screens

Main Screen

```
*IDLE Shell 3.13.0*
File Edit Shell Debug Options Window Help
Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: K:\@Ak\@temp\IP temp\Project Work IP\Main.Project.IP.XII.2025.aks242424\hotel_management_system.py

CONGRATULATIONS! Your SQLite connection has been established!

1. Add New Customer
2. Make a Booking
3. Room Rent
4. Restaurant Bill
5. Gaming Bill
6. Fashion Bill
7. Show Total Bill
8. Search Customer
9. Exit

Enter your choice: |
```

CUSTOMER DETAILS

```
*IDLE Shell 3.13.0*
File Edit Shell Debug Options Window Help
Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC
Type "help", "copyright", "credits" or "license()" for more info
>>>
= RESTART: K:\@Ak\@temp\IP temp\Project Work IP\Main.Project.IP.

CONGRATULATIONS! Your SQLite connection has been established!

1. Add New Customer
2. Make a Booking
3. Room Rent
4. Restaurant Bill
5. Gaming Bill
6. Fashion Bill
7. Show Total Bill
8. Search Customer
9. Exit

Enter your choice: 1

1. Add New Customer
9. Exit

Enter your choice: 1
Enter Customer Identification Number: C101
Enter Customer Name: Harry Potter
Enter Customer Address: New York
Enter Customer Age: 29
Enter Customer Country: America
Enter Customer Contact Number: 212-456-7890
Enter Customer Email: harry_potter90@gmail.com

New Customer entered in the system successfully!
```


CUSTOMER BOOKING RECORDS

```
*IDLE Shell 3.13.0*
File Edit Shell Debug Options Window Help
Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (AMD64)]
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: K:\@Ak\@temp\IP temp\Project Work IP\Main.Project.IP.XII.2025.aks242424\h
CONGRATULATIONS! Your SQLite connection has been established!

1. Add New Customer
2. Make a Booking
3. Room Rent
4. Restaurant Bill
5. Gaming Bill
6. Fashion Bill
7. Show Total Bill
8. Search Customer
9. Exit

Enter your choice: 2
Enter customer details to make a booking.
Enter Customer ID or Name: C101

#### We have the following rooms for you ####
1. Ultra Royal > 10000 Rs. per day
2. Royal > 5000 Rs. per day
3. Elite > 3500 Rs. per day
4. Budget > 2500 Rs. per day
5. Exit
Enter your choice: 4
Enter Start Date (YYYY/MM/DD): 2024/11/12
Enter End Date (YYYY/MM/DD): 2024/11/17
Thank you, your room has been booked from 2024/11/12 to 2024/11/17 for 5 days.
Your total booking price is: Rs. 12500
```

ROOM RENT

```
*IDLE Shell 3.13.0*
File Edit Shell Debug Options Window Help
9. Exit

Enter your choice: 3
Enter customer details to calculate room rent.
Enter Customer ID or Name: 1
Customer not found!

1. Add New Customer
2. Make a Booking
3. Room Rent
4. Restaurant Bill
5. Gaming Bill
6. Fashion Bill
7. Show Total Bill
8. Search Customer
9. Exit

Enter your choice: 3
Enter customer details to calculate room rent.
Enter Customer ID or Name: C101

#### We have the following rooms for you ####
1. Ultra Royal > 10000 Rs.
2. Royal > 5000 Rs.
3. Elite > 3500 Rs.
4. Budget > 2500 Rs.
5. Exit
Enter your choice: 3
Enter Customer Room No: 201
Enter No. of Days: 3
Thank you, your room has been booked for: 3 days
Your total room rent is: Rs. 10500
```

RESTAURENT BILL

```
Enter your choice: 4
Enter customer details for restaurant bill.
Enter Customer ID or Name: C101
1. Vegetarian Combo > 300 Rs.
2. Non-Vegetarian Combo > 500 Rs.
3. Vegetarian & Non-Vegetarian Combo > 750 Rs.
4. Exit
Enter your choice: 2
Enter Quantity: 2
Your total bill amount is: Rs. 1000
```

**** WE HOPE YOU WILL ENJOY YOUR MEAL ****

GAMING BILL

```
Enter your choice: 5
Enter customer details for gaming bill.
Enter Customer ID or Name: C101
1. Virtual Reality > 1500 Rs.
2. Arcade > 1000 Rs.
3. Racing > 2000 Rs.
4. Escape Room > 2500 Rs.
5. Exit
Enter your choice: 1
Enter Game Time (in hours): 2
Your total gaming bill is: Rs. 3000
```

FASHION STORE BILL

```
Enter your choice: 6
Enter customer details for fashion bill.
Enter Customer ID or Name: C101
1. T-Shirt > 500 Rs.
2. Jeans > 1000 Rs.
3. Jacket > 1500 Rs.
4. Shoes > 2000 Rs.
5. Exit
Enter your choice: 1
Enter Quantity: 1
Your total fashion bill is: Rs. 500
```

TOTAL BILL

```
Enter your choice: 7
Enter customer details to show total bill.
Enter Customer ID or Name: C101
```

```
***** FINAL BILL *****
Customer ID: C101
=====
Room Rent: Rs. 10500
Restaurant Bill: Rs. 1000
Gaming Bill: Rs. 3000
Fashion Bill: Rs. 500
=====
Total Amount: Rs. 15000
*****
```

1. Add New Customer
2. Make a Booking
3. Room Rent
4. Restaurant Bill
5. Gaming Bill
6. Fashion Bill
7. Show Total Bill
8. Search Customer
9. Exit

```
Enter your choice: |
```

DETAILS OF THE CUSTOMER

```
Enter your choice: 8
Enter Customer ID or Name: C101
```

```
Customer found:
ID: C101
Name: Harry Potter
Address: New York
Age: 29
Country: America
Contact Number: 212-456-7890
Email: harry_potter90@gmail.com
```

1. Add New Customer
2. Make a Booking
3. Room Rent
4. Restaurant Bill
5. Gaming Bill
6. Fashion Bill
7. Show Total Bill
8. Search Customer
9. Exit

```
Enter your choice: |
```

EXIT

```
Enter your choice: 8
Enter Customer ID or Name: C101
```

```
Customer found:
ID: C101
Name: Harry Potter
Address: New York
Age: 29
Country: America
Contact Number: 212-456-7890
Email: harry_potter90@gmail.com
```

1. Add New Customer
2. Make a Booking
3. Room Rent
4. Restaurant Bill
5. Gaming Bill
6. Fashion Bill
7. Show Total Bill
8. Search Customer
9. Exit

```
Enter your choice: 9
```

```
Thank you for using the system!
```

```
>>> |
```

MYSQL DATABASE AND TABLES USED IN THIS PROJECT

DATABASE

	name
1	C_DETAILS
2	ROOM_BOOKINGS
3	ROOM_RENT
4	RESTAURANT
5	GAMING
6	FASHION

C_DETAILS TABLE STRUCTURE

Table: C_DETAILS		
Column	Data type	Constraints
CID	TEXT	•PRIMARY KEY
C_NAME	TEXT	
C_ADDRESS	TEXT	
C_AGE	INT	
C_COUNTR Y	TEXT	
P_NO	INT	
C_EMAIL	TEXT	

ROOM_BOOKINGS TABLE STRUCTURE

Table: ROOM_BOOKINGS		
Column	Data type	Constraints
CID	TEXT	
ROOM_CHOICE	INT	
START_DATE	TEXT	
END_DATE	TEXT	
TOTAL_PRICE	INT	
Global table constraints		
•FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)		

ROOM_RENT TABLE STRUCTURE

Table: ROOM_RENT		
Column	Data type	Constraints
CID	TEXT	
ROOM_CHOICE	INT	
NO_OF_DAYS	INT	
ROOM_NO	INT	
ROOM_RENT	INT	
Global table constraints		
•FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)		

RESTAURANT TABLE STRUCTURE

Table: RESTAURANT		
Column	Data type	Constraints
CID	TEXT	
CUISINE	TEXT	
QUANTITY	INT	
BILL	INT	
Global table constraints		
•FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)		

GAMING TABLE STRUCTURE

Table: GAMING		
Column	Data type	Constraints
CID	TEXT	
GAME_NAME	TEXT	
GAME_TIME	INT	
BILL	INT	
Global table constraints		
•FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)		

FASHION TABLE STRUCTURE

Table: FASHION		
Column	Data type	Constraints
CID	TEXT	
ITEM_NAME	TEXT	
QUANTITY	INT	
BILL	INT	
Global table constraints		
•FOREIGN KEY(CID) REFERENCES C_DETAILS(CID)		

BACKEND DATA GENERATED THROUGH SOFTWARE

Table: C_DETAILS					
#	CID TEXT	C_NAME TEXT	C_ADDRESS TEXT	C_AGE INT	C_COUNTRY TEXT
0	C101	Harry Potter	New York	28	America
1	C103	Naruto Uzumaki	Konoha	19	Hidden Leaf Village
2	C105	Monkey D Luffy	Foosha Village	19	Dawn Island
3	C106	Tsunade Senju	Konoha	55	Land of Fire

BACKEND DATA GENERATED THROUGH SOFTWARE

Table: ROOM_BOOKINGS					
#	CID TEXT	ROOM_CHOICE INT	START_DATE TEXT	END_DATE TEXT	TOTAL_PRICE INT
0	C101	2	2024/10/12	2024/10/24	60000
1	C105	3	2024/10/15	2024/10/20	21000
2	C101	4	2024/11/01	2024/11/10	25000
3	C106	1	2024/12/05	2024/11/13	90000

BACKEND DATA GENERATED THROUGH SOFTWARE

Table: ROOM_RENT					
#	CID TEXT	ROOM_CHOICE INT	NO_OF_DAYS INT	ROOM_NO INT	ROOM_RENT INT
0	C101	1	2	201	20000
1	C103	4	4	205	10000
2	C106	2	1	301	5000
3	C105	3	5	105	17500

BACKEND DATA GENERATED THROUGH SOFTWARE

Table: RESTAURANT				
#	CID TEXT	CUISINE TEXT	QUANTITY INT	BILL INT
0	C101	2	2	1000
1	C103	1	1	300
2	C105	1	2	600
3	C106	3	3	2250

BACKEND DATA GENERATED THROUGH SOFTWARE

Table: GAMING				
#	CID TEXT	GAME_NAM E TEXT	GAME_TIM E INT	BILL INT
0	C101	1	2	3000
1	C103	4	1	2500
2	C105	3	2	4000
3	C106	2	1	1000

BACKEND DATA GENERATED THROUGH SOFTWARE

Table: FASHION				
#	CID TEXT	ITEM_NAM E TEXT	QUANTIT Y INT	BILL INT
0	C101	4	2	4000
1	C103	3	1	1500
2	C105	1	3	1500
3	C106	2	2	2000

Hardware and Software Requirements

1. Hardware Requirement

- Pentium 3/4/core 2 Duo/Dual core/i3/i5/i7
- With at least 256 MB RAM
- 5MB free space on Hard Disk
- Color Monitor/LCD

2. Software Requirement

- MS Windows
- Python with related library used for data analysis
 - Pandas
 - Numpy
 - Matplotlib
- SQ lite for RDBMS.